The First Eye: Stage Three of a Verified Reader, Movement One

Working paper v0.1, Project secret3, August 11, 2026

Abstract

Stage three of the C5 campaign is the internalization: a term inside the calculus that meets the stage-two engine's specification. This paper delivers the campaign's first movement: the workshop bench that makes internal machines provable, and the first verified detector built on it. The bench is three lemmas. Substitution ignores absent names (sub1NoOcc); a substituted abstraction still beta-reduces, into the composed substitution, under one side condition (absSubSteps); and the packaged two-argument beta for the vanishing engine, abs2Beta, whose side condition, the second argument is free of the first name, is discharged in every use by stage one's discipline theorem quoteClosed, which here becomes load-bearing rather than decorative. On the bench: Scott-style result tags inside the calculus, nothing and just, with their case behaviors; the internal constructor of quoted applications, qMk, with its beta theorem; and the eye itself, readK, a closed term that inspects any quoted program and answers whether its root is a K redex. Seven compiled theorems cover every code exactly: readKFires delivers just the kept child on the redex shape, and six silence theorems deliver nothing on the six non-firing shapes, matching the K clause of the engine clause for clause. Every proof is a composition of named lemmas; the longest chain is six links, and no raw position bookkeeping occurs anywhere except a two-step discard. The remaining movements are specified.

1. The movement plan

The full internal reader must case on a quote, detect redexes, assemble contracta, and recurse. Each capability is a movement: this paper delivers detection for K, with the bench that all later movements share; the S eye and the assembler are movement two, the same pattern one level deeper plus qMk; the dispatcher that unifies the eyes is movement three; and the knot, recursion through the fixed point with correctness across its unfoldings, is the honestly hard remainder and is named as such. The order is dictated by proof economy: everything before the knot is compositional over the bench, and the bench is where the design decisions live.

2. The bench, and one design decision

The decision that makes the movement cheap: all machine bodies are concrete terms over concrete variable names. Then substitution into them is computation, the checker evaluates it, and per-instance equalities are definitional; only two general lemmas are ever needed. The first, sub1NoOcc, says substitution for an absent name is the identity, by the occurrence ledger. The second, absSubSteps, is the movement's real content: applying a substituted abstraction still walks to the doubly substituted body,

```
absSubSteps : occ x v = 0 ->
  Steps (App (sub1 x u (abs y b)) v) (sub1 x u (sub1 y v b))
```

proved by the same induction that proved absCorrect, case for case, because sub1 distributes over the engine's S and K skeleton. The packaged corollary abs2Beta chains absCorrect with it and is the only beta lemma any two-argument machine needs. Its side condition is exactly where stage one

pays off: every argument a reader ever receives is a quote, quotes are closed by quoteClosed, and the condition discharges by that theorem at every one of the six sites in this paper.

3. Tags and the constructor

Results are Scott-Maybe inside the calculus: nothing is K itself, two symbols; just x is a seven-symbol selector; their case behaviors, nTagCase and jTagCase, are one and six moves. The internal constructor qMk takes two quotes and builds the quote of the application, and its theorem qMkBeta is a single abs2Beta instance, because the quote of an application is, by stage one's design, the workshop pair wrapped twice, a concrete body. The assembler of movement two is qMk used twice.

4. The eye

readK is three nested abstractions with concrete branch terms: case the quote; atoms answer nothing; the application branch receives both children and cases the left child; its atoms answer nothing; its application branch receives the grandchildren and cases the left grandchild; K there means the root was a K redex, answer just the kept grandchild; everything else swallows its arguments and answers nothing. Seven theorems then cover the code space exactly, one per shape, quantified over all subcodes:

```
readKFires : Steps (readK (quote (QApp (QApp QK a) y))) (jTag (quote a))
```

and readKAtomS, readKAtomK, readKAppS, readKAppK, readKAppSS, readKDeep, each ending in nothing. The firing proof is six lemma links: the engine's beta, the alphabet's application case, the bench's two-argument beta, the alphabet again one level down, the bench again, and the alphabet's K case delivering the tag. The silence proofs are prefixes and variants of the same chain. Nothing in any of them is a raw move except the two-step discard drop2Case; the module's earlier layers do all the walking.

5. The road, and the knot named

Movement two, the S eye: identical pattern one level deeper, with the contractum assembled by two uses of qMkBeta, quote (QApp (QApp a g) (QApp b g)) from the three quoted cargos; nothing in it requires new bench lemmas. Movement three, the dispatcher: compose the eyes with the tag cases into a single rootStep term agreeing with the engine's root clauses, again pure composition. Movement four is the knot: descending into non-root positions requires the term to call itself, which in this calculus means the fixed point, and correctness must then track the specification through the knot's own unfoldings interleaved with every chain above. That is the summit's true face, it is not underestimated here, and everything below it is now standing.

Appendix A: Methods

All theorems are in the shared proof module Proofs.idr, 2,453 lines after this paper's additions, %default total, no postulates, no holes; idris2 —check Proofs.idr is the entire verification, per the series' standing methods statement.

Appendix B: The first-eye section (module part T)

```
--------------------------------------------------------------------
-- Tier 2, part T: the first eye (C5 stage three, movement one)
--------------------------------------------------------------------

||| a variable absent from a term is untouched by substitution
sub1NoOcc : (x : String) -> (u, t : Term) ->
```

```
  occ x t = 0 -> sub1 x u t = t
sub1NoOcc x u (Var y) prf with (x == y)
  sub1NoOcc x u (Var y) prf | True impossible
  sub1NoOcc x u (Var y) prf | False = Refl
sub1NoOcc x u CK prf = Refl
sub1NoOcc x u CS prf = Refl
sub1NoOcc x u (App a b) prf =
  rewrite sub1NoOcc x u a (fst (plusZero (occ x a) (occ x b) prf)) in
  rewrite sub1NoOcc x u b (snd (plusZero (occ x a) (occ x b) prf)) in
  Refl
  where
    plusZero : (m, n : Nat) -> m + n = 0 -> (m = 0, n = 0)
    plusZero Z n prf = (Refl, prf)
    plusZero (S m) n prf impossible

||| a substituted abstraction still beta-reduces, into the composed
||| substitution, provided the second argument is free of the first name
absSubSteps : (x, y : String) -> (b, u, v : Term) ->
  occ x v = 0 ->
  Steps (App (sub1 x u (abs y b)) v) (sub1 x u (sub1 y v b))
absSubSteps x y (Var z) u v pv with (y == z)
  absSubSteps x y (Var z) u v pv | True =
    rewrite sub1NoOcc x u v pv in skIdentity CK v
  absSubSteps x y (Var z) u v pv | False with (x == z)
    absSubSteps x y (Var z) u v pv | False | True = kTrue u v
    absSubSteps x y (Var z) u v pv | False | False = kTrue (Var z) v
absSubSteps x y CK u v pv = kTrue CK v
absSubSteps x y CS u v pv = kTrue CS v
absSubSteps x y (App a b) u v pv =
  Then RedS $
  stepsTrans (appLs (absSubSteps x y a u v pv))
             (stepsAppR (absSubSteps x y b u v pv))

||| two-argument beta for the vanishing engine, closed second argument
abs2Beta : (x, y : String) -> (b, u, v : Term) ->
  occ x v = 0 ->
  Steps (App (App (abs x (abs y b)) u) v)
        (sub1 x u (sub1 y v b))
abs2Beta x y b u v pv =
  stepsTrans (appLs (absCorrect x (abs y b) u))
             (absSubSteps x y b u v pv)

||| tagged results inside the calculus: nothing and just
nTag : Term
nTag = CK

jTag : Term -> Term
jTag x = App CK (App (App CS Proofs.iC) (App CK x))

||| nothing selects its first continuation
nTagCase : (n, j : Term) -> Steps (App (App Proofs.nTag n) j) n
nTagCase n j = kTrue n j

||| just x hands x to its second continuation
jTagCase : (x, n, j : Term) ->
```

```
  Steps (App (App (Proofs.jTag x) n) j) (App j x)
jTagCase x n j =
  Then (CtxL RedK) $
  Then RedS $
  Then (CtxL (CtxL RedS)) $
  Then (CtxL (CtxL (CtxL RedK))) $
  Then (CtxL RedK) $
  Then (CtxR RedK) Done

||| the internal constructor of quoted applications
qMk : Term
qMk = abs "u" (abs "v"
        (App CK (App CK (Proofs.pairC (Var "u") (Var "v")))))

qMkBeta : (x, y : Code) ->
  Steps (App (App Proofs.qMk (Proofs.quote x)) (Proofs.quote y))
        (Proofs.quote (QApp x y))
qMkBeta x y =
  abs2Beta "u" "v"
    (App CK (App CK (Proofs.pairC (Var "u") (Var "v"))))
    (quote x) (quote y) (quoteClosed "u" y)

||| the two-argument discard: swallow both, answer nothing
drop2 : Term
drop2 = App CK (App CK Proofs.nTag)

drop2Case : (s, t : Term) -> Steps (App (App Proofs.drop2 s) t) Proofs.nTag
drop2Case s t = Then (CtxL RedK) $ Then RedK Done

||| the first eye: does the root of this quoted program fire as K?
gTerm : Term
gTerm = abs "p" (abs "q"
  (App (App (App (Var "p") Proofs.nTag) (Proofs.jTag (Var "q")))
       Proofs.drop2))

fTerm : Term
fTerm = abs "l" (abs "r"
  (App (App (App (Var "l") Proofs.nTag) Proofs.nTag) Proofs.gTerm))

readK : Term
readK = abs "q0"
  (App (App (App (Var "q0") Proofs.nTag) Proofs.nTag) Proofs.fTerm)

||| the eye fires: a quoted K redex yields just its kept child
readKFires : (a, y : Code) ->
  Steps (App Proofs.readK (Proofs.quote (QApp (QApp QK a) y)))
        (Proofs.jTag (Proofs.quote a))
readKFires a y =
  stepsTrans (absCorrect "q0"
    (App (App (App (Var "q0") Proofs.nTag) Proofs.nTag) Proofs.fTerm)
    (quote (QApp (QApp QK a) y))) $
  stepsTrans (qAppCase (QApp QK a) y nTag nTag fTerm) $
  stepsTrans (abs2Beta "l" "r"
    (App (App (App (Var "l") Proofs.nTag) Proofs.nTag) Proofs.gTerm)
    (quote (QApp QK a)) (quote y) (quoteClosed "l" y)) $
```

```
  stepsTrans (qAppCase QK a nTag nTag gTerm) $
  stepsTrans (abs2Beta "p" "q"
    (App (App (App (Var "p") Proofs.nTag) (Proofs.jTag (Var "q")))
         Proofs.drop2)
    (quote QK) (quote a) (quoteClosed "p" a)) $
  qKCase nTag (jTag (quote a)) drop2

||| and stays silent on every other shape
readKAtomS : Steps (App Proofs.readK (Proofs.quote QS)) Proofs.nTag
readKAtomS =
  stepsTrans (absCorrect "q0"
    (App (App (App (Var "q0") Proofs.nTag) Proofs.nTag) Proofs.fTerm)
    (quote QS)) $
  qSCase nTag nTag fTerm

readKAtomK : Steps (App Proofs.readK (Proofs.quote QK)) Proofs.nTag
readKAtomK =
  stepsTrans (absCorrect "q0"
    (App (App (App (Var "q0") Proofs.nTag) Proofs.nTag) Proofs.fTerm)
    (quote QK)) $
  qKCase nTag nTag fTerm

readKAppS : (y : Code) ->
  Steps (App Proofs.readK (Proofs.quote (QApp QS y))) Proofs.nTag
readKAppS y =
  stepsTrans (absCorrect "q0"
    (App (App (App (Var "q0") Proofs.nTag) Proofs.nTag) Proofs.fTerm)
    (quote (QApp QS y))) $
  stepsTrans (qAppCase QS y nTag nTag fTerm) $
  stepsTrans (abs2Beta "l" "r"
    (App (App (App (Var "l") Proofs.nTag) Proofs.nTag) Proofs.gTerm)
    (quote QS) (quote y) (quoteClosed "l" y)) $
  qSCase nTag nTag gTerm

readKAppK : (y : Code) ->
  Steps (App Proofs.readK (Proofs.quote (QApp QK y))) Proofs.nTag
readKAppK y =
  stepsTrans (absCorrect "q0"
    (App (App (App (Var "q0") Proofs.nTag) Proofs.nTag) Proofs.fTerm)
    (quote (QApp QK y))) $
  stepsTrans (qAppCase QK y nTag nTag fTerm) $
  stepsTrans (abs2Beta "l" "r"
    (App (App (App (Var "l") Proofs.nTag) Proofs.nTag) Proofs.gTerm)
    (quote QK) (quote y) (quoteClosed "l" y)) $
  qKCase nTag nTag gTerm

readKAppSS : (a, y : Code) ->
  Steps (App Proofs.readK (Proofs.quote (QApp (QApp QS a) y))) Proofs.nTag
readKAppSS a y =
  stepsTrans (absCorrect "q0"
    (App (App (App (Var "q0") Proofs.nTag) Proofs.nTag) Proofs.fTerm)
    (quote (QApp (QApp QS a) y))) $
  stepsTrans (qAppCase (QApp QS a) y nTag nTag fTerm) $
  stepsTrans (abs2Beta "l" "r"
    (App (App (App (Var "l") Proofs.nTag) Proofs.nTag) Proofs.gTerm)
```

```
    (quote (QApp QS a)) (quote y) (quoteClosed "l" y)) $
  stepsTrans (qAppCase QS a nTag nTag gTerm) $
  stepsTrans (abs2Beta "p" "q"
    (App (App (App (Var "p") Proofs.nTag) (Proofs.jTag (Var "q")))
         Proofs.drop2)
    (quote QS) (quote a) (quoteClosed "p" a)) $
  qSCase nTag (jTag (quote a)) drop2

readKDeep : (p, q, b, y : Code) ->
  Steps (App Proofs.readK (Proofs.quote (QApp (QApp (QApp p q) b) y)))
        Proofs.nTag
readKDeep p q b y =
  stepsTrans (absCorrect "q0"
    (App (App (App (Var "q0") Proofs.nTag) Proofs.nTag) Proofs.fTerm)
    (quote (QApp (QApp (QApp p q) b) y))) $
  stepsTrans (qAppCase (QApp (QApp p q) b) y nTag nTag fTerm) $
  stepsTrans (abs2Beta "l" "r"
    (App (App (App (Var "l") Proofs.nTag) Proofs.nTag) Proofs.gTerm)
    (quote (QApp (QApp p q) b)) (quote y) (quoteClosed "l" y)) $
  stepsTrans (qAppCase (QApp p q) b nTag nTag gTerm) $
  stepsTrans (abs2Beta "p" "q"
    (App (App (App (Var "p") Proofs.nTag) (Proofs.jTag (Var "q")))
         Proofs.drop2)
    (quote (QApp p q)) (quote b) (quoteClosed "p" b)) $
  stepsTrans (qAppCase p q nTag (jTag (quote b)) drop2) $
  drop2Case (quote p) (quote q)
```

References

1. Project secret3, ‘A Quoted Alphabet’, working paper v0.1, and ‘The Reader’s Engine’, working paper v0.1, August 11, 2026.
2. Project secret3, ‘The Vanishing Engine, Verified’, module part N with chapter two of Rise of Form, August 2026.
3. Project secret3, ‘Every Typed Song Ends’, working paper v0.1, August 11, 2026, whose mechanization notes this movement’s bench continues.
4. Project secret3, research agenda v0.10, line C5.